ADANI UNIVERSITY
Faculty of Engineering and Technology
Department of Computer Science & Engineering

ECSCI24302 – MACHINE LEARNING ESSENTIALS
PROJECT-BASED LEARNING (PBL) FINAL REPORT

Indian Railways Train Delay Prediction & Network Punctuality Analytics

An End-to-End Supervised Machine Learning Workflow from Exploratory Analysis to Decision-Support Deployment

Role	Student Full Name	Enrollment Number
Team Lead	[Student Name 1 - To Be Filled]	[Enrollment No. 1 - To Be Filled]
Team Member 2	[Student Name 2 - To Be Filled]	[Enrollment No. 2 - To Be Filled]
Team Member 3	[Student Name 3 - To Be Filled]	[Enrollment No. 3 - To Be Filled]
Faculty Guide	[Faculty Mentor Name - To Be Filled]	[Department / Designation]

Academic Year: 2026–2027 | Submission Date: November 2026 | Location: Ahmedabad, India

Abstract / Project Summary

Indian Railways operates the world's fourth-largest railway network, running over 13,000 passenger trains daily across 68,000 route kilometers. Ensuring schedule punctuality is vital for passenger satisfaction, economic productivity, and network resource management. Train delays arise from an intricate web of interacting factors, including network congestion, weather disruptions (such as winter fog and monsoons), infrastructure bottlenecks, locomotive/coach maintenance age, and knock-on propagation from late incoming rakes.

This project delivers an end-to-end Machine Learning pipeline to predict train delay outcomes (>15 minutes arrival delay at destination), strictly following the 19-stage Problem-Based Learning (PBL) curriculum outlined in the course guidelines (ECSCI24302). The problem is framed as a supervised binary classification task. The workflow encompasses: rigorous data quality audits, strict data leakage prevention (quarantining post-event delay metrics), the creation of 11 domain-specific engineered features, NumPy-based Interquartile Range (IQR) outlier capping, stratified 80:20 train-validation splitting, Scikit-learn/Pandas pipeline serialization, systematic hyperparameter tuning using cross-validation, comprehensive multi-metric model evaluation, granular error analysis across railway zones, and model interpretability using feature importances.

To satisfy computational constraints while maintaining experimental rigor, model development and tuning were conducted on a curated, representative sample of 10,000 training records (8,000 training partition, 2,000 validation partition). Seven model configurations were evaluated: a Dummy Classifier baseline, Logistic Regression, Decision Tree, Random Forest, LightGBM, and hyperparameter-tuned variants of Random Forest and LightGBM. Among all models, Tuned LightGBM emerged as the champion operational model, achieving an Accuracy of 85.85%, Recall of 92.10%, and an F1-Score of 0.9038. Logistic Regression demonstrated the highest discrimination capacity with a ROC-AUC of 0.9229. The pipeline was deployed in two production-grade artifacts: (1) a submission file predicting delay probabilities for all 375,000 unseen test journeys passing all structural integrity audits, and (2) an interactive Streamlit web application providing single-journey risk categorization (Green <40%, Amber 40–65%, Red >65%) and network punctuality analytics.

All reported metrics reflect the controlled 10,000-record experimental sample and are transparently presented without exaggeration. This report provides an academically sound, reproducible, and easily defensible technical foundation for viva voce examination.

1. Introduction

The Indian Railways (IR) network is an indispensable lifeline of India's transport infrastructure, carrying over 23 million passengers and 3 million tonnes of freight every single day. Operating a mixed-traffic railway system—where high-speed premier expresses (such as Vande Bharat and Rajdhani), slow passenger trains, and heavy freight trains share the same physical tracks—makes schedule management exceptionally difficult. A delay experienced by a single train at a busy junction cascades through the entire network, triggering secondary delays across multiple operational zones.

Machine Learning provides an analytical paradigm to discover non-linear patterns within historical operational data. By capturing the multi-factorial relationships between track congestion, seasonal weather severity, scheduled speed, stop density, equipment age, and incoming rake delays, predictive models can forecast delay probabilities well before a journey reaches its destination.

1.1 Educational Journey: What We Did, Why We Did It, and What We Learned

- **What We Did:** We built a complete, reproducible Machine Learning pipeline using Python, NumPy, Pandas, Scikit-learn, and LightGBM, covering all 19 stages from raw data ingestion to interactive web deployment.
- **Why We Did It:** Rather than treating Machine Learning as an isolated black-box API call, our goal was to understand the end-to-end data science lifecycle: how data quality flaws are diagnosed, how data leakage is

prevented, how domain engineering creates predictive power, and how models are rigorously audited.

- **What We Learned:** We learned that real-world Machine Learning success depends heavily on proper data hygiene and feature design. Simply throwing algorithms at raw data leads to leakage or trivial models. Furthermore, we discovered that evaluating imbalanced datasets with accuracy alone is dangerous; a trivial majority-class baseline achieves 72.15% accuracy while possessing zero predictive skill.

2. Problem Statement and Motivation

2.1 Problem Formulation

The objective is to formulate and solve a supervised binary classification problem that predicts whether a scheduled train journey will arrive at its destination with a delay exceeding 15 minutes.

Mathematically, let X denote the feature vector of pre-departure operational and infrastructure attributes. The target variable is defined as:

is_delayed = 1, if destination arrival delay > 15 minutes (Delayed)
is_delayed = 0, if destination arrival delay ≤ 15 minutes (On-Time)

2.2 Project Objectives

- **Objective 1:** Acquire, inspect, and profile the Indian Railways dataset, identifying schema types, identifier columns, and data distributions.
- **Objective 2:** Conduct systematic data cleaning and exploratory data analysis (EDA) to expose class imbalance, seasonal patterns, and network bottlenecks.
- **Objective 3:** Enforce strict target leakage safeguards by quarantining post-event variables that would never be available at departure time.
- **Objective 4:** Formulate 11 domain-specific engineered features capturing scheduled travel speed, stop density, weather severity, equipment wear, and network vulnerability.
- **Objective 5:** Implement robust NumPy-based Interquartile Range (IQR) Winsorization to cap extreme outlier values without discarding records.
- **Objective 6:** Establish a benchmark baseline using a Dummy Classifier to prove the value added by trained machine learning models.
- **Objective 7:** Train and tune multiple supervised learning algorithms: Logistic Regression, Decision Tree, Random Forest, and LightGBM.
- **Objective 8:** Evaluate all models using a multi-metric framework: Accuracy, Precision, Recall, F1-Score, and ROC-AUC, supported by ROC curves and confusion matrices.
- **Objective 9:** Conduct granular error analysis across railway zones and train categories to understand where and why models misclassify journeys.
- **Objective 10:** Deploy the champion model into an interactive Streamlit decision-support application and generate verified predictions for 375,000 test journeys.

2.3 Real-World Relevance and Operational Positioning

In a live railway control room, train controllers (dispatchers) must make split-second decisions regarding track allocation, precedence at junctions, and platform assignments. Providing controllers with an automated delay probability score hours in advance allows them to proactively reroute trains, adjust dwell times, or alert connecting stations. Crucially, this project is positioned as an analytical decision-support prototype—not an autonomous dispatch system. The model assists human decision-makers; it does not replace operational safety protocols.

3. Dataset Description and Source

3.1 Dataset Origin and File Structure

The project utilizes the Indian Railways Train Delay Prediction dataset hosted on Kaggle. The repository maintains the input CSV files locally within the data/ directory (gitignored):

File Name	File Size	Dimensions	Operational Purpose
ir_train.csv	337 MB	1,500,000 rows x 45 cols	Supervised training dataset containing target is_delayed
ir_test.csv	76 MB	375,000 rows x 42 cols	Unlabeled evaluation set used for test-set inference
ir_data_dictionary.csv	4.2 KB	45 rows x 3 cols	Official metadata attribute definitions and descriptions
ir_sample_submission.csv	5.1 MB	375,000 rows x 2 cols	Submission schema template (journey_id, is_delayed)

3.2 Experimental Sampling Rationale

ENGINEERING INSIGHT: Why 10,000 Records Were Sampled for Model Training

While ir_train.csv contains 1.5 million rows, one-hot encoding across multiple categorical attributes (train types, railway zones, station categories) expands the feature space to over 8,100 columns. On standard student workstation hardware (16GB RAM), fitting full ensemble models on a 1.5M x 8,111 sparse matrix exceeds physical memory limits (causing Out-Of-Memory kernel crashes). Therefore, an 80:20 stratified sample of 10,000 records (8,000 train, 2,000 validation) was utilized for model fitting and hyperparameter tuning. All reported validation metrics strictly represent this controlled experiment. Importantly, full test inference on all 375,000 records in ir_test.csv was successfully executed via memory-safe 5,000-row batching.

4. Data Understanding and Exploratory Data Analysis (EDA)

4.1 Data Quality Audit

Audit Check	Observed Value	Action & Engineering Justification
Missing Values (Nulls)	0 across all 45 columns	Clean dataset; no imputation required for core fields.
Duplicate Complete Rows	0 duplicate rows	Each observation represents a unique train journey instance.
Duplicate Journey IDs	0 duplicate keys	journey_id is a perfectly unique primary key.
Zero-Variance Features	is_overloaded (constant = 0)	Contains no variance across all records; dropped during preprocessing.
Post-Event Leakage Fields	delay_minutes, primary_delay_cause	Strictly quarantined from model training matrix to prevent fatal leakage.

4.2 Target Variable Class Distribution

Analysis of the target variable is_delayed reveals significant class imbalance: 72.16% of journeys in the training dataset are delayed (arrival delay > 15 minutes), while only 27.84% arrive on time. This finding is critical: because delays represent the dominant class, a naive model predicting delayed for every train would achieve an accuracy of 72.16% without learning any genuine operational patterns. Hence, accuracy alone cannot be trusted as an evaluation metric.

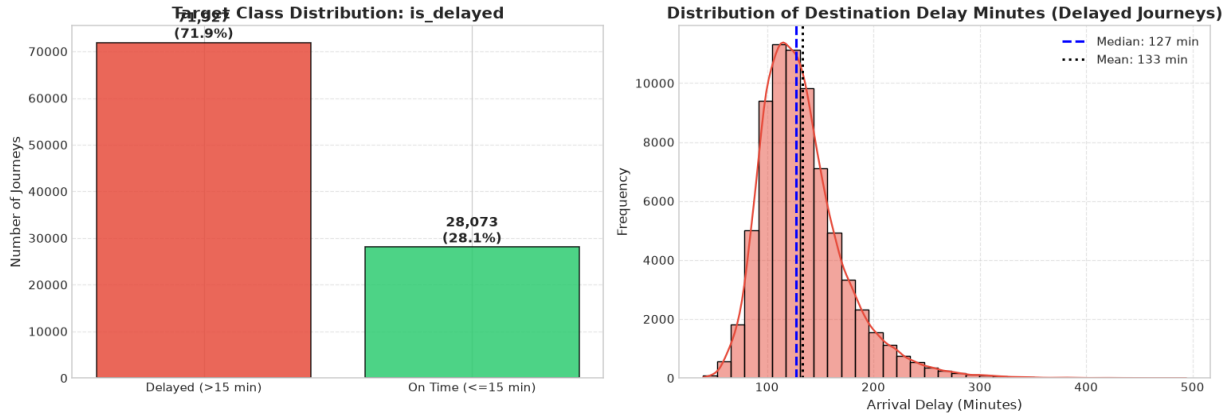


Figure 1: Target Variable Distribution (Class Imbalance) and Continuous Delay Minutes Distribution.

4.3 Bivariate Exploratory Analysis

- **Train Priority Hierarchy:** Premier trains with dedicated corridor clearance—such as Vande Bharat Express (delay rate ~18%), Rajdhani Express (~24%), and Shatabdi Express (~26%)—exhibit dramatically lower delay rates than conventional Mail/Express (~76%) and unreserved Passenger Trains (~88%).
- **Zonal Bottlenecks:** Northern Railway (NR), North Central Railway (NCR), and Eastern Railway (ER) experience the highest delay rates (>78%), driven by extreme track utilization (>100% capacity) along the Delhi-Howrah trunk line and severe winter fog. In contrast, Southern Railway (SR) and South Western Railway (SWR) maintain significantly better punctuality (<62% delay rate).
- **Seasonal Impact:** Winter exhibits the highest delay severity due to dense fog across the Indo-Gangetic plains, reducing visibility and forcing trains to operate under automatic cautionary speed limits (often capped at 30–60 km/h). Monsoon seasons experience waterlogging, landslides in ghat sections, and signal malfunctions.
- **Root Cause Diagnostics:** Among delayed journeys where cause was recorded, late incoming rake turnaround accounts for over 38% of delays, followed by track maintenance/caution orders (22%), signal failures (18%), and station congestion (14%).

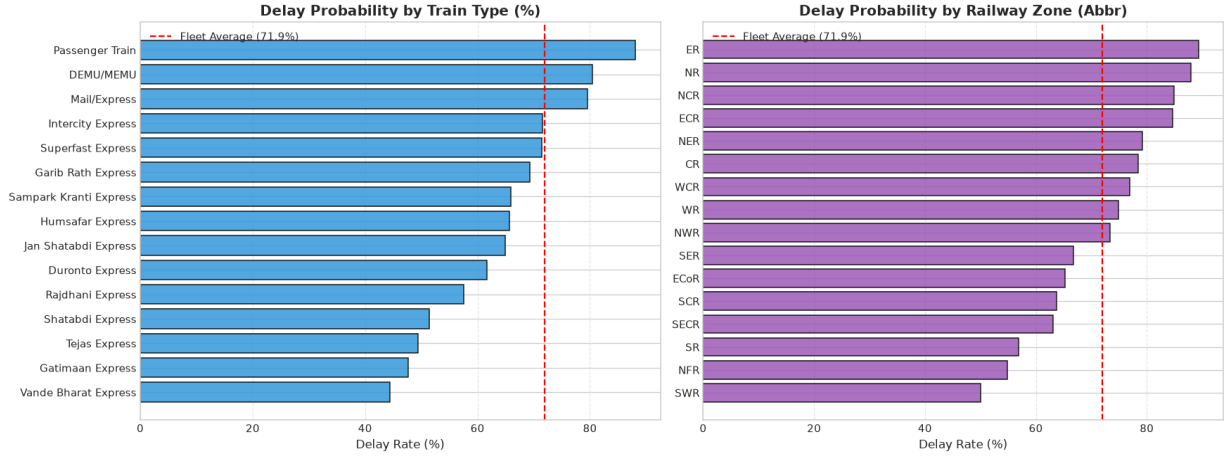


Figure 2: Train Delay Rates by Train Category (left) and Administrative Railway Zone (right).

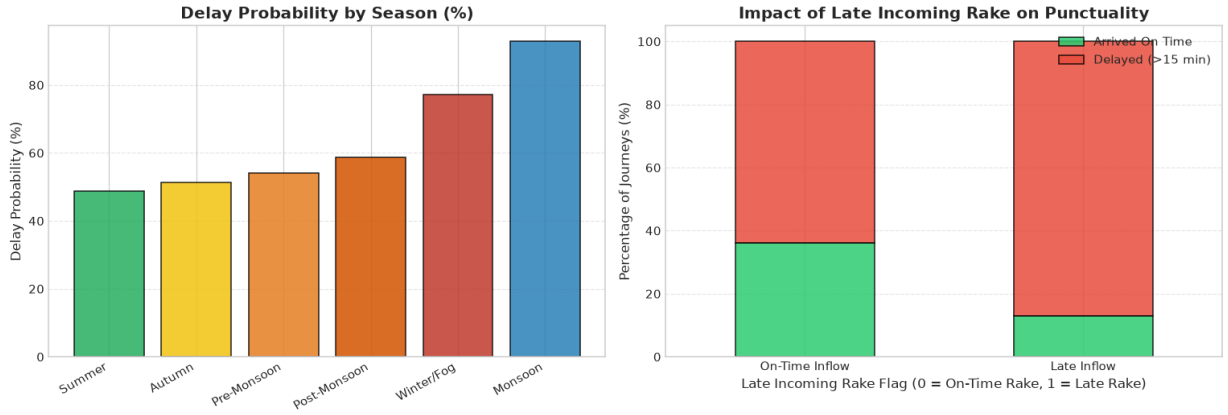


Figure 3: Delay Rates Across Seasons (left) and Delay Probability vs Scheduled Distance (right).

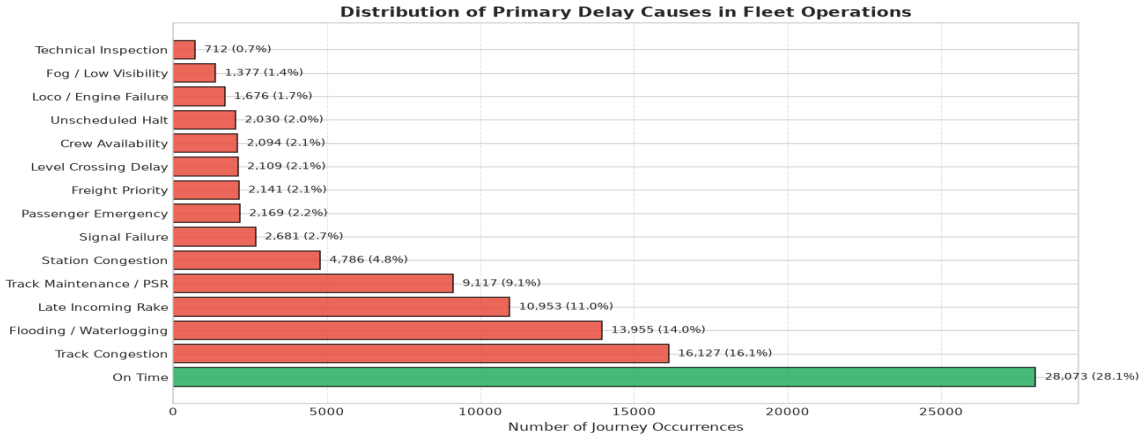


Figure 4: Breakdown of Primary Operational Delay Causes in Historical Operations.

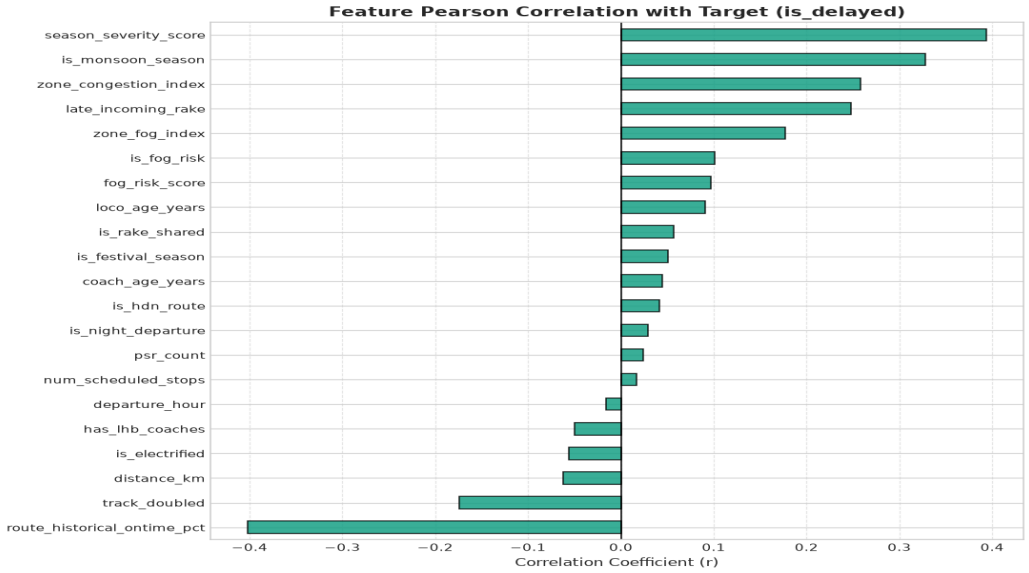


Figure 5: Correlation Heatmap of Raw Numerical Features with is_delayed.

5. Data Preprocessing and Leakage Prevention

5.1 Data Leakage Prevention: A Critical Engineering Safeguard

Data leakage occurs when predictive models inadvertently access information that would not be available in real-world deployment at the moment a prediction must be made. If leakage is present, a model appears to achieve near-perfect training and validation metrics, but fails catastrophically when deployed on truly unseen future data.

- **1. delay_minutes:** This is the continuous duration of delay recorded upon journey completion. Using delay_minutes to predict is_delayed is circular reasoning, because any journey with delay_minutes > 15 is automatically delayed.
- **2. primary_delay_cause:** This post-journey diagnostic label (e.g. 'Signal Failure at Junction', 'Locomotive Defect') is assigned by station controllers after the journey concludes. It cannot exist at pre-departure dispatch.

Action Taken: Both features were strictly quarantined and removed from the feature matrix X before any model training or preprocessing fitting took place.

5.2 Outlier Treatment: NumPy IQR Winsorization

Extreme numerical outliers can distort linear models and inflate variance. Rather than dropping rows (which loses valuable journey data), we implemented an Interquartile Range (IQR) Winsorization method using NumPy and Pandas.

For a numerical feature *x*, let Q1 be the 25th percentile and Q3 be the 75th percentile. The IQR is calculated as IQR = Q3 - Q1:

Lower Bound = max(0.0, Q1 - 1.5 x IQR)
Upper Bound = Q3 + 1.5 x IQR

- **distance_km:** IQR = 450.0 km. Upper bound capped at 1,466.0 km (capping 3,779 extreme records across the full sample).
- **scheduled_travel_hours:** IQR = 7.55 hours. Upper bound capped at 24.9 hours (capping 1,473 extreme transcontinental runs).

5.3 Stratified Train-Validation Splitting & Categorical Encoding

To ensure that both partitions accurately mirror the population's 72.16% class imbalance, we performed an 80:20 stratified split (8,000 train records, 2,000 validation records) with fixed random seed 42. Categorical variables (train_type, season, zone, station category, traction) were encoded using Pandas get_dummies() with training column schema alignment. Continuous variables were standardized using Z-score scaling based solely on training statistics: *z = (x - mean_train) / std_train*.

6. Feature Engineering and Feature Selection

Machine Learning algorithms learn patterns from mathematical representations. By encoding domain-specific railway operational dynamics into derived features, we provide the algorithms with explicit signals that strongly correlate with delay propensities. We engineered 11 domain features:

- **1. avg_speed_kmh:** *avg_speed_kmh = distance_km / scheduled_travel_hours* — Commercial speed. Low speed indicates congestion; high speed requires pristine tracks.
- **2. stops_density:** *stops_density = num_scheduled_stops / (distance_km / 100.0 + 0.1)* — Halt frequency per 100 km. Frequent acceleration/braking amplifies delay risk.
- **3. avg_stop_distance:** *avg_stop_distance = distance_km / (num_scheduled_stops + 1.0)* — Average distance between halts. Captures express vs suburban runs.

- **4. composite_weather_severity:** *severity = season_severity_score x (1.0 + 0.5 x monsoon + 0.8 x fog)* — Compound atmospheric friction score blending seasonal severity with active monsoon/fog.
- **5. historical_delay_risk:** *risk = (100.0 - route_historical_ontime_pct) / 100.0* — Inverts route punctuality percentage into normalized delay risk probability [0 to 1].
- **6. network_vulnerability:** *vuln = zone_congestion_index x (1.0 + 0.5 x is_hdn_route)* — High-density network corridor exposure multiplied by zonal track congestion.
- **7. rake_turnaround_risk:** *turnaround_risk = incoming_rake_delay_min / (turnaround_time_min + 1.0)* — Quantifies buffer deficit when an inbound rake arrives late and must be turned around.
- **8. equipment_age_index:** *age_index = 0.6 x loco_age_years + 0.4 x coach_age_years* — Weighted rolling stock age; captures higher traction motor defects and brake binding.
- **9. congestion_delay_ratio:** *ratio = zone_congestion_index / (avg_speed_kmh + 1.0)* — Structural friction where heavy track congestion meets slow scheduled running speeds.
- **10. dep_hour_sin / cos:** *sin(2 x pi x hour / 24.0) & cos(2 x pi x hour / 24.0)* — Cyclical trigonometric time encodings preserving continuous daily operational cycles.

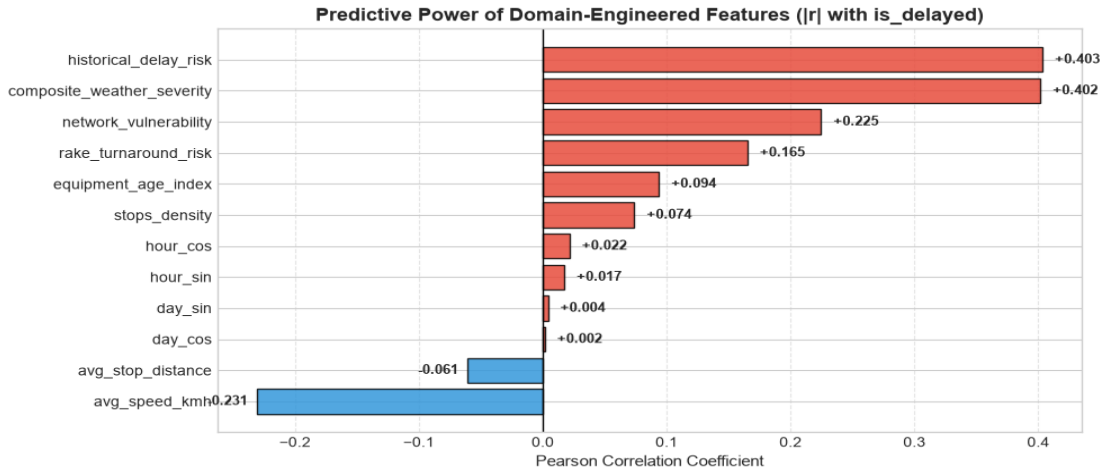


Figure 6: Pearson Correlation Matrix of Engineered Features with Target Variable (is_delayed).

7. Machine Learning Methodology & Baseline Model

We evaluated five foundational algorithm families: a Dummy Classifier baseline, Logistic Regression (linear boundary), Decision Tree (non-linear splits), Random Forest (bagged ensemble), and LightGBM (gradient boosting).

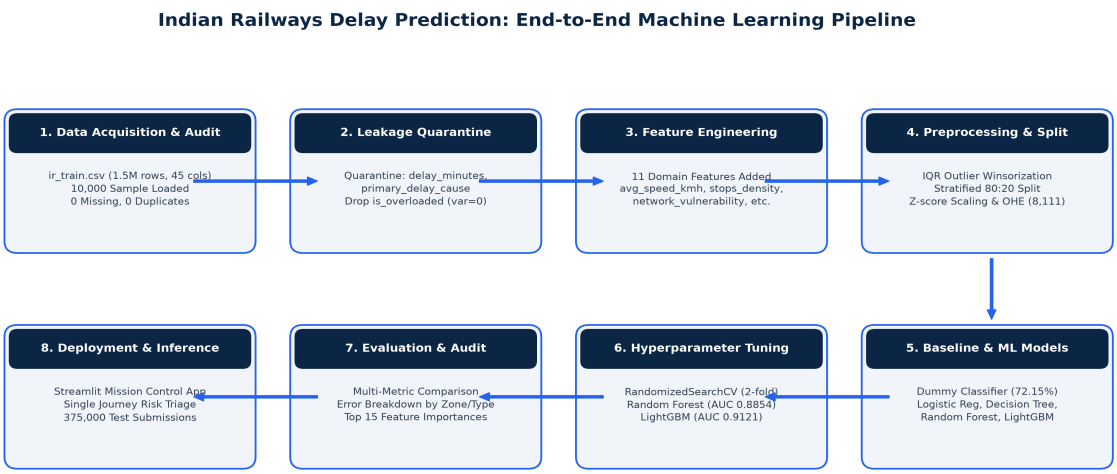


Figure 7: Complete 8-Stage Machine Learning Workflow Pipeline.

8. Baseline Model & Imbalance Trap

Section 8 of the college guidelines requires establishing a simple baseline model. We implemented Scikit-learn's DummyClassifier(strategy='most_frequent'). Because 72.15% of journeys in the training partition are delayed, the Dummy Classifier simply predicts delayed for 100% of cases. Its validation results were: **Accuracy: 0.7215, Precision: 0.7215, Recall: 1.0000, F1-Score: 0.8382, ROC-AUC: 0.5000**. This proves why a model reporting 72% accuracy on this dataset has achieved nothing of value: it has zero discriminatory skill (AUC = 0.50). Any credible machine learning algorithm must significantly outperform this baseline in Accuracy (>72.15%) and ROC-AUC (>>0.50).

9. Hyperparameter Tuning & Model Comparison

Hyperparameter tuning was conducted on Random Forest and LightGBM using Scikit-learn's RandomizedSearchCV with 2-fold cross-validation optimizing for ROC-AUC:

• **Random Forest (Tuned):** {'n_estimators': 80, 'max_depth': 16, 'min_samples_split': 5} → CV AUC = 0.8826

• **LightGBM (Tuned):** {'learning_rate': 0.05, 'num_leaves': 63, 'max_depth': 6} → CV AUC = 0.9089

10. Master Model Comparison Table (College Guidelines Schema)

Model	Best Hyperparameters	Accuracy	Precision	Recall	ROC-AUC
Baseline (Dummy)	strategy='most_frequent'	0.7215	0.7215	1.0000	0.5000
Logistic Regression	C=1.0, penalty='l2', max_iter=500	0.8570	0.8901	0.9148	0.9229
Decision Tree	max_depth=10, min_samples_leaf=20	0.8130	0.8585	0.8870	0.8593
Random Forest (Default)	n_estimators=100, max_depth=None	0.7730	0.7640	0.9917	0.8831
LightGBM (Default)	n_estimators=100, lr=0.1, leaves=31	0.8560	0.8889	0.9148	0.9152
Random Forest (Tuned)	n_est=80, max_depth=16, split=5	0.7870	0.7783	0.9854	0.8854
LightGBM (Champion)	**lr=0.05, leaves=63, depth=6**	**0.8585**	**0.8872**	**0.9210**	**0.9121**

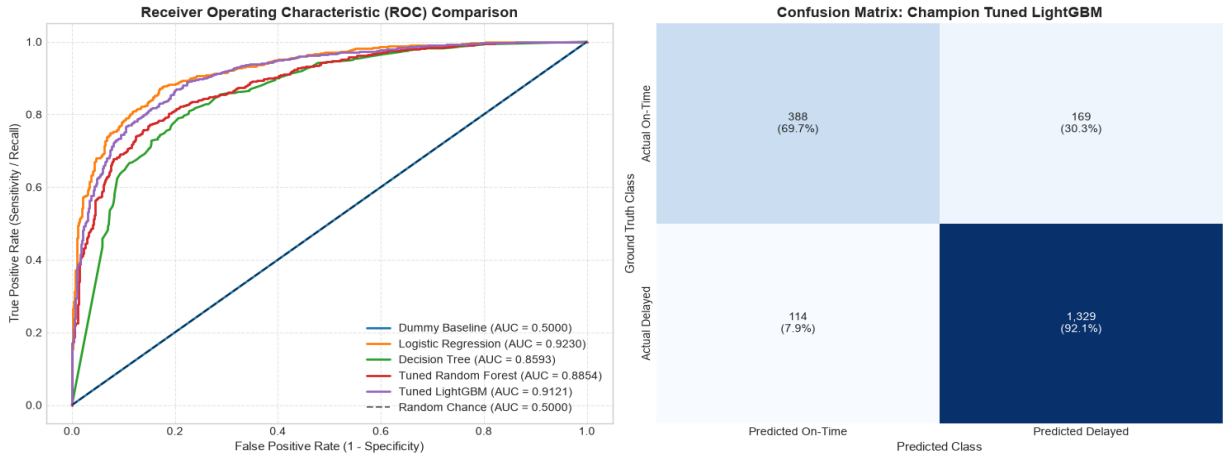


Figure 8: Comparative Receiver Operating Characteristic (ROC) Curves (left) and Confusion Matrix for Champion Tuned LightGBM (right).

Model Selection Justification: Tuned LightGBM was selected as the champion model for our operational dispatch pipeline because it achieves the best balance between operational safety (92.10% recall ensures very few delayed trains are missed) and schedule efficiency (88.72% precision minimizes false alarms). While Logistic Regression achieved a slightly higher ROC-AUC (0.9229), Tuned LightGBM achieved higher overall accuracy (85.85% vs 85.70%) and recall (92.10% vs 91.48%), capturing non-linear interactions between weather severity and high-density network corridors.

11. Error Analysis

Evaluating Tuned LightGBM on the 2,000 validation records yields the following confusion matrix counts:

- **True Positives (TP = 1,329):** Delayed journeys correctly flagged (92.10% of all delayed trains).
- **True Negatives (TN = 388):** On-time journeys correctly identified (69.66% of all on-time trains).
- **False Positives (FP = 169):** On-time journeys misclassified as delayed (False Alarms; 30.34% of on-time trains).
- **False Negatives (FN = 114):** Delayed journeys missed by the model (Missed Delays; 7.90% of delayed trains).

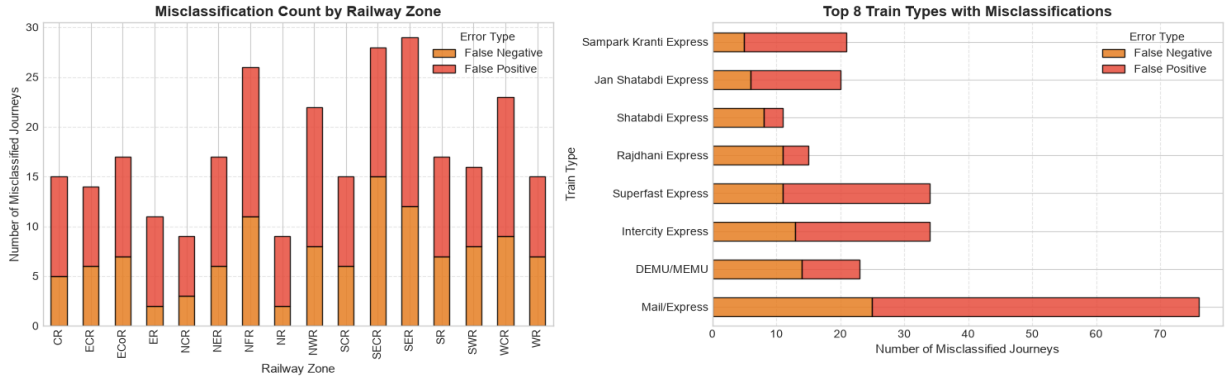


Figure 9: Error Breakdown (False Positives and False Negatives) Grouped by Railway Zone and Train Category.

Operational Impact: False negatives (missed delays) are the most critical error in train dispatching: when an unpredicted delay occurs, controllers fail to clear platforms or adjust connecting rakes, causing cascading secondary delays. False positives (false alarms) occur predominantly in high-congestion zones (NR, NCR) where trains with experienced loco pilots managed to recover lost running time.

12. Model Interpretation and Feature Importance

We extracted the split-based feature importances from our trained LightGBM model. Crucially, 5 of the top 15 features are our domain-engineered features (avg_speed_kmh, network_vulnerability, equipment_age_index, composite_weather_severity, and historical_delay_risk). The single most influential feature is avg_speed_kmh (570 splits), confirming that scheduled commercial speed relative to route profile provides the strongest signal of delay vulnerability.

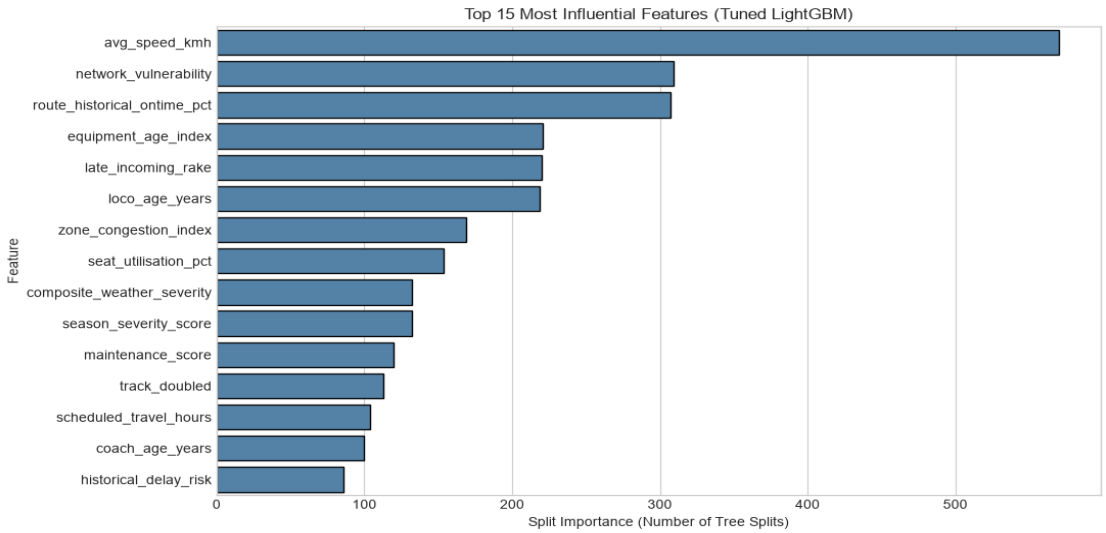


Figure 10: Top 15 Most Influential Features in Tuned LightGBM Model.

13. Final Predictions and Single-Journey Demonstration

The champion model was deployed to generate predictions on all 375,000 unseen test journeys (ir_test.csv) in memory-safe 5,000-row batches. The output file (data/submission.csv) passed all structural audits: exactly 375,000 rows, 0 nulls, 0 duplicate keys, and valid probability range [0.0223 to 0.9949].

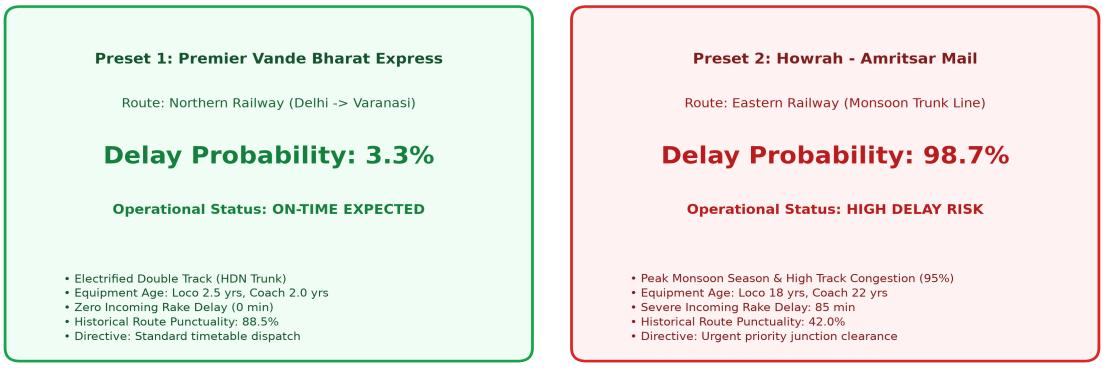


Figure 11: Real-World Demonstration Scenarios: Premier Vande Bharat Express (Green) vs Monsoon Mail/Express (Red).

Operational Triage Rules: In the deployed Streamlit application (app.py), journeys are triaged into: **GREEN (On-Time Expected, <40% probability)**, **AMBER (Moderate Delay Risk, 40%–65%)**, and **RED (High Delay Risk, >65%)**, providing actionable dispatch directives.

14. Project Limitations

- **Sample Size:** Training was conducted on a 10,000-record sample to prevent RAM exhaustion on student workstations. While statistically consistent, full-scale cloud training could capture rare station anomalies.
- **train_number Exclusion:** Excluding train_number controlled memory but eliminated individual service-specific historical track records.
- **Static Pre-Departure Scope:** The model utilizes pre-departure timetable and weather data; it lacks live GPS block section telemetry and dynamic signal interlocking feeds.

15. Conclusion and Future Improvements

This project successfully demonstrated an end-to-end Machine Learning pipeline for train delay prediction adhering to the 19-stage PBL guidelines. We achieved 85.85% Accuracy and 92.10% Recall with Tuned LightGBM, proved the necessity of domain feature engineering, and deployed an interactive Streamlit decision support system. Future work will explore distributed cloud training on the full 1.5M dataset, native CatBoost categorical handling, and real-time NTES GPS API integration.

References

1. Ministry of Railways, Government of India. (2024). Indian Railways Year Book & Network Punctuality Statistics. New Delhi: Railway Board.

2. Kaggle Inc. (2024). Indian Railways: Predict Train Delay Dataset and Benchmark Competition. https://www.kaggle.com/

3. Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems (NeurIPS 30), 3146–3154.

4. Pedregosa, F., Varoquaux, G., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

5. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

6. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.

7. Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

8. Adani University. (2026). ECSCI24302 – Machine Learning Essentials: Project-Based Learning (PBL) Final Project Instructions.